Digital Signal Processing Laboratory ELEN 133L Spring 2025

April 10, 11, 2025

Dylan Thornburg & Alan Rieger & Ayden Dauenhauer

Laboratory 2: DTMF Tones in Touch-Tone Telephone Dialing: Signal Generation

Prelab due Wednesday April 9 by 8 pm

Objectives:

- Use MATLAB to create a function.
- Design and test a function to create a telephone dialing signal for a specified telephone number.
- Compute the frequency response of two filters using the MATLAB function *freqz* and observe the effect of two filters on the dialing signal.

Laboratory:

Step 1: Create and debug a function to generate a DTMF signal as described in the preLab

- Follow your planned flowchart and write your m-file to implement the function *my_dtmf*. Save it in a file called **my_dtmf.m** in your MATLAB working directory. You will be able to call this function from your live script in the same way that you can use other MATLAB functions such as *sin* and *cos*.
    - Implementation notes:
      a.) It is not necessary to specify the length of dial_vals. Your function can compute the length using: **num=length(dial_vals);**

      b.) The output vector can be computed for each button value and concatenated to form the output signal:
      dial_sig = [ ]; initializes the output to an empty vector
      dial_sig = [dial_sig new_sig quiet_sig]; concatenates the vector new_sig, computed for a single button, and quiet _sig, a vector of zeros for the quiet interval, with the current value of dial_sig.

      c.) The vector new_sig can be computed for each button value using the cosine function as in the previous laboratory. Frequencies assigned to each button can be implemented with two vectors – one for row

frequencies and one for column frequencies, or a case statement, or if-then-else.

d.) The dial_vals vector can use 11 to represent 0 because it is “button 11” or 0 can be detected and handled differently from the other numeric values. Be sure to consider what you want your function to do if an element of dial-vals is not one of the 12 expected possibilities.

e.) The vector quiet_sig can be computed once using
quiet_sig = zeros(1, fs*quiet_time);

f.) After all the dial_vals digits have been included in your output signal vector, be sure to normalize the signal so the maximum absolute magnitude is 1.

g.) Note that comments immediately following the first line function declaration in your m-file will be printed whenever you type “help my_dtmf” at the
MATLAB “>>” prompt. For your function, write comments that will be helpful.

- After you have saved your function as an m-file in your working directory, type ‘help my_dtmf” in the command window to read your comments describing how the function works.
- Create a live script to use your function. In Section 1, the function will be tested.
    - Set your parameters for a sampling frequency of 8000, a tone time of 0.5 seconds, a quiet time of 0.1 seconds and a dial value vector with a single number.
    - Write a statement that creates an output signal vector by calling your function with your four parameter values.
    - Run section 1 and debug the m-file function until there are no errors.
- When the function runs without detected errors: Add the following to Section 1:

    - Use sound to listen to your signal. Does it sound right? Compare it to a telephone when the same button is pressed. Be sure that you divide the output signal by the maximum value so that the output values will be between -1 and +1. Otherwise the nonlinear clipping will create additional frequencies in the sound.

      Yes our signal sounds right in comparison to a real phone signal.
    - Plot your signal. Create a time vector that is the same length as your

signal and has time steps of 1/(sampling frequency) so that you can plot your signal as a function of time. Describe the display of the complete signal. Verify that the tone time is correct and that the amplitude does not exceed 1.

We see a high frequency signal with amplitudes that max out at one. We also can see our quiet_tone spaces where there are no signals being plotted.

- Use the zoom feature of the figure display to show regions that are about 400 samples wide. (Or use an axis to do this.) Describe the zoomed-in display. Does it look like the sum of two sinusoids that have no harmonic relationship? An example for comparison is shown below.

  The zoomed-in display looks like a bunch of sine waves and yes, they appear to not have any Harmonic relationship.

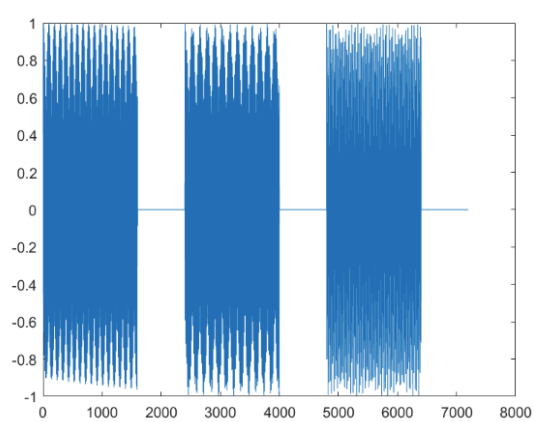
- Change the dial value vector to be a vector of length 2 which contains two different numbers. Run Section 1 again. Verify that the quiet time is correct.

  Yes, the quiet time is correct.

Checkpoint 1: Demonstrate your function to your laboratory assistant for three different input numbers specified by the lab assistant.

Plot of [3, 6, 11] -> (3, 6, 0)

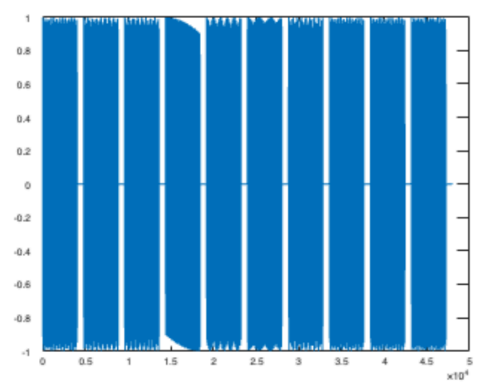
```
x = my_dtmf(.2, .1, 8000, [3, 6, 11]);
plot(x)
sound(x)
```



Step 2: Test and explore the DTMF signal generated by your function.

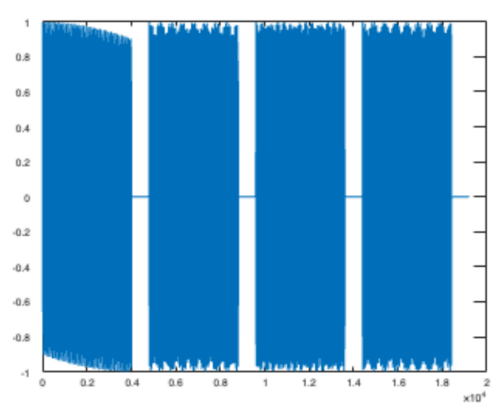
- Add Section 2 to set your four function parameters.
  - Set the sampling frequency to 8000 Hz. Set the tone time to 0.5 seconds, the quiet time to 0.1 seconds.
  - Set the dial value vector to a phone number you use.
  - Run Section 2 and verify that the values are correct by looking at the workspace window.
- Add Section 3 to call your function and create an output signal.
  - Create an output signal vector by calling your function using the four inputs set in Section 2.
  - Plot your signal as a function of time.
  - Use sound to listen to your signal. Explicitly include the sampling frequency in the sound function call.

```
x = ELEN133L_Lab2(0.5, 0.1, 8000, [4, 11, 8, 3, 9, 1, 2, 4, 4, 6]);

plot(x)
sound(x)
```



- Change Section 2 parameters. Make the sampling frequency 8000, the tone time 0.5 seconds, the quiet time 0.1 seconds and the dial value vector = [3,5,7,11]. Run Section 2.
- Run Section 3 to create the new signal, plot it, and listen to it.
  - Include the plot in your report. Verify that the tone time, the quiet time and the total time are correct.

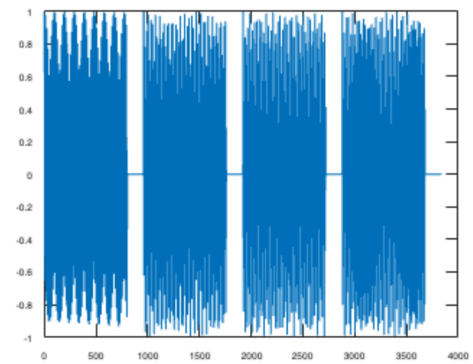
```
x = ELEN133L_Lab2(0.5, 0.1, 8000, [3,5,7,11]);

plot(x)
sound(x)
```



- Change Section 2 to reduce the duration of the signal. Change the tone time to 0.1 and the quiet time to 0.02. Run Section 2.
- Run Section 3 to create the new signal, plot it, and listen to it.
    - Include the plot in your report. Verify that the tone time, the quiet time and the total time are correct.
    - Compare the sound of this signal to the previous signal with a longer duration.

The signal was faster with the same amount of numbers passed into the function.

```
x = ELEN133L_Lab2(0.1, 0.02, 8000, [3,5,7,11]);

plot(x)
sound(x)
```



Checkpoint 2: Show your two saved plots to your laboratory assistant.

Step 3: Explore the effect of the sampling frequency.

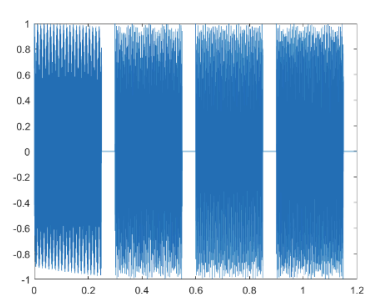
- Change Section 2 parameters. Make the sampling frequency <u>8000</u>, the tone time 0.25 seconds, the quiet time 0.05 seconds and the dial value vector = [3,5,7,11]. Run Section 2.
- Run Section 3 and remember the sound produced. Record the length of your output signal vector from the workspace window and the total time duration of your signal from the plot.

Our output signal vector is 9600 samples long. The total time duration of our signal is 1.2 seconds (including a quiet time after the last tone).

Live Editor - Z:\ELEN133\Lab2\Lab2_P3.mlx

my_dtmf.m | Lab2_P1.mlx | Lab2_P2.mlx | Lab2_P3.mlx

```
tone_len = .25;
quiet_len = .05;
Fs = 8000;
n = 4;
dT = 1/8000;

x = my_dtmf(tone_len, quiet_len, Fs, [3, 5, 7, 11]);

n_real = n * (tone_len + quiet_len);

tvec = (0: (n_real * Fs -1) ) * dT;
plot(tvec, x)
sound(x)
```



- In Section 2, change the sampling frequency to <u>16000</u>, but do not change the

other three parameters. Run Section 2.

- Run Section 3.
    - If it does not sound the same as the previous sound output, check to be sure you included the sampling frequency when you called the sound function.
    - Record the length of your output signal vector from the workspace window and the total time duration of your signal from the plot. How do they compare to the values when the sampling frequency was 8000?

Our output signal vector is 19200 samples long, but still 1.2 seconds long. There is double the amount of samples but the time is still the same. You get the same values at Fs =16000 versus Fs = 8000 but they are every other value since you get intermediate values for Fs = 16000.

Fs = 8000:

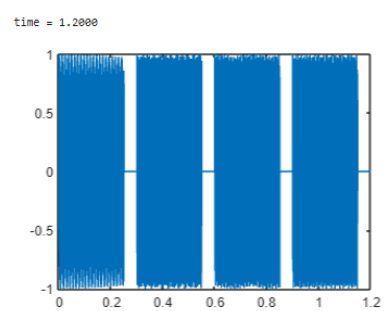
0.626589049643884 -0.111460060727317 -0.507331020907403 -0.326164113137355 -0.0168553871347183 -0.105176896610232 -0.517064521871381 -0.658232074128797 -0.157201844889072 0.629406545484350 0.973617890042787

Fs = 16000:

0.899613967598960 **0.626589049643884** 0.256472720304580 **-0.111460060727317** -0.385056872970591 **-0.507331020907403** -0.472681444627271 **-0.326164113137355** -0.146243867808426 **-0.0168553871347183** 0.00191878551764602

BOLD: Same Values in both Fs = 8000 and Fs = 16000

```
clear all
fs=16000;
dT=1/fs;
tone_len=0.25;
quiet_len=0.05;
n = 4;
x = my_dtmf(tone_len, quiet_len, fs, [3, 5, 7, 11]);
time = n*(tone_len+quiet_len)
tvec = (0:(time*fs-1))*dT;
sound(x, fs)
plot(tvec, x)
```



- In Section 2, change the sampling frequency to 4000, but do not change the other three parameters. Run Section 2.
- Run Section 3.
    - Compare this sound to the sound produced when the sampling frequency

was 8000. Describe what you would expect to hear.

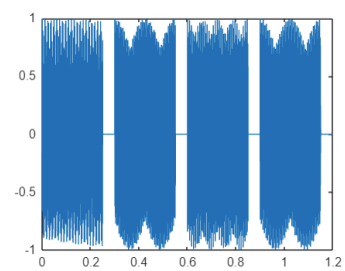
The sound still has enough samples to sound roughly the same at Fs = 4000 Hz versus Fs = 8000 Hz.

- Record the length of your output signal vector from the workspace window and the total time duration of your signal from the plot. How do they compare to the values when the sampling frequency was 8000?

Now there are half the amount of samples (still 1.2 seconds). Every other value isn't sampled for the 4000 hz sampling frequency. Thus, Fs = 8000 will have one intermediate value that Fs = 4000 will not have.

Lab2_P3.mlx | my_dtmf.m

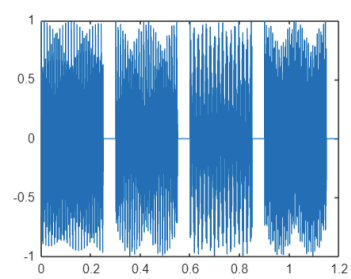
```
tone_len = .25;
quiet_len = .05;
Fs = 4000;
n = 4;
dT = 1/Fs;

x = my_dtmf(tone_len, quiet_len, Fs, [3, 5, 7, 11]);

time = n * (tone_len + quiet_len);

tvec = (0: (time * Fs -1) ) * dT;
plot(tvec, x)
sound(x, Fs)
```



- In Section 2, change the sampling frequency to 2000, but do not change the other three parameters. Run Section 2.
- Run Section 3.
  - Compare this sound to the sound produced when the sampling frequency was 8000. Describe what you would expect to hear.
  - Record the length of your output signal vector from the workspace window and the total time duration of your signal from the plot. How do they compare to the values when the sampling frequency was 8000?

This time we lost enough data that the signal sounded different. The time is still the same, but the samples are 1/4 of 8000's total samples (2000). Thus, Fs = 8000 will have 3 intermediate values in between each value that is the same in Fs = 2000.
For some of our numbers' signals, we see them start to develop these spikes at lower sampling frequencies. This is due to aliasing. Aliasing occurs because we are sampling at a rate lower than twice the highest frequency present in the signal.

```
tone_len = .25;
quiet_len = .05;
Fs = 2000;
n = 4;
dT = 1/Fs;

x = my_dtmf(tone_len, quiet_len, Fs, [3, 5, 7, 11]);

time = n * (tone_len + quiet_len);

tvec = (0: (time * Fs -1) ) * dT;
plot(tvec, x)
sound(x, Fs)
```



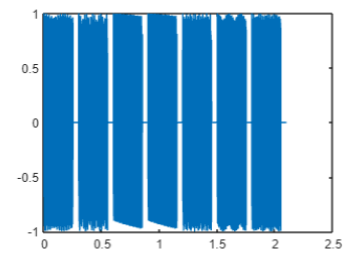
Checkpoint 3: Demonstrate the sounds to your laboratory assistant.

Step 4: Observe the effect of filtering on the dial signals.

- Change Section 2 parameters. Make the sampling frequency 8000, the tone time 0.25 seconds, the quiet time 0.05 seconds and the dial value vector = [1 2 3 3 6 9 12]. Run Section 2.
- Run Section 3
    - Remember the sound produced.
    - Save the plot to include in your report.

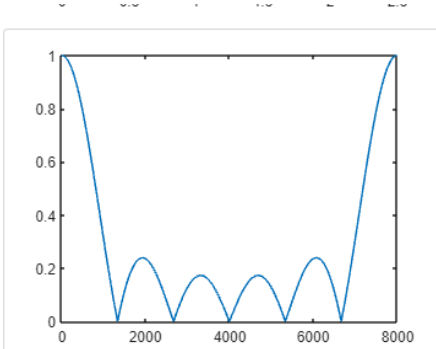
```
clear all
fs=8000;
dT=1/fs;
tone_len=0.25;
quiet_len=0.05;
n = 7;
x = my_dtmf(tone_len, quiet_len, fs, [1, 2, 3, 3, 6, 9, 12]);
time = n*(tone_len+quiet_len)
tvec = (0:(time*fs-1))*dT;
sound(x, fs)
plot(tvec, x)
```

```
time = 2.1000
```



- Add Section 4 to define the filter using the two coefficient vectors **a** and **b**.
    - Create coefficient vectors for a moving average filter of length 6. The **a** vector will just be the scalar value 1, and the **b** vector will have a length of 6 and all elements will have the value 1/6.
    - Compute the frequency response of the filter and plot its magnitude using [H,fv] = freqz(b,a,'whole',2048, fs); % fs is the sampling frequency. plot(fv, abs(H))
    - Run Section 4.
    - Include the frequency response plot in your report.

```
a = 1;
b = [1/6, 1/6, 1/6, 1/6, 1/6, 1/6];
[H,fv] = freqz(b,a,'whole',2048, fs);
plot( fv, abs(H));
```



    - Examine the plot and record the magnitude of the frequency response at each of the row frequencies and column frequencies.

Frequency (Hz)	Magnitude of the Frequency Response
697	0.61287
770	0.543639
852	0.460374
941	0.367946
1209	0.103538
1336	0.00204168
1477	0.100721

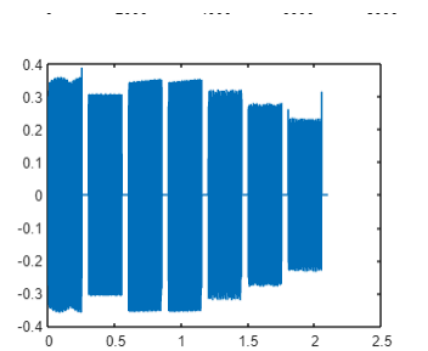
- Predict the effect you think the filter will have on each of the dial signals from each of the buttons specified in the dial value vector.

We think it will act like a moving average filter which will look like both a low and high pass filter (band stop filter) and average out sampled values so they appear lower.

- Add Section 5
  - Use the MATLAB function *filter* to create the filtered output signal when the input is a dial signal that was created in Section 3 and the filter is defined by the a and b vectors in Section 4.
  - Plot the filter output as a function of time and include the plot in your report. Is the plot consistent with your predictions?

```
a = 1;
b = [1/6, 1/6, 1/6, 1/6, 1/6, 1/6,];
[H,fv] = freqz(b,a,'whole',2048, fs);
plot( fv, abs(H));
```

```
y = filter(b, a, x);
plot(tvec, y);
```

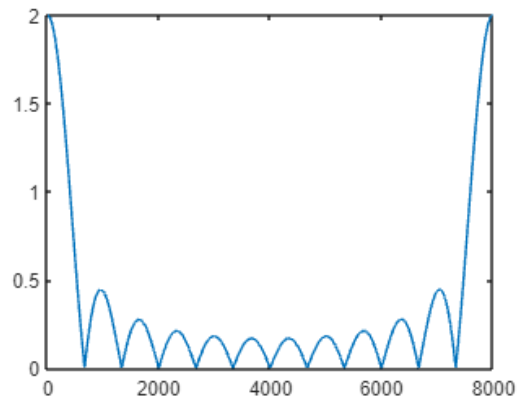


Yes, the plot is consistent with our predictions since the amplitude is lower than 0.4 when it was previously 1 since it should decrease.

  - Use sound to listen to the filter output. Is the sound consistent with your predictions?

Yes, the filter made the sound quieter on the parts that were damped as seen in the plot.

- Change Section 4 to define a moving average filter of length 12 instead of 6 and rerun Section 4.
    - Include the frequency response plot in your report.



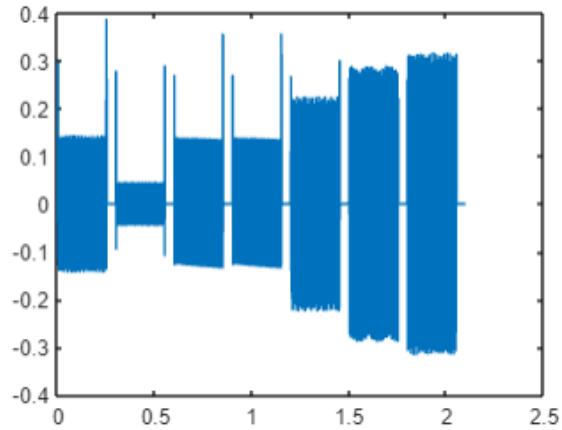
    - Examine the plot and record the magnitude of the frequency response at each of the row frequencies and column frequencies.

Frequency (Hz)	Magnitude of the Frequency Response
697	0.0939209
770	0.26095
852	0.388556
941	0.443792
1209	0.198925
1336	0.00408328
1477	0.190079

    - Predict the effect you think the filter will have on each of the dial signals from each of the buttons specified in the dial value vector.

The filter will dampen some sounds even more now because we average from more points.

- Rerun Section 5
    - Include the plot in your report. Is the plot consistent with your predictions?



Yes, the plot is consistent with our predictions since the signals are much more damped than the moving average of filter length 6 as seen in the magnitude of frequency response table.

- Is the sound consistent with your predictions?

Yes, certain buttons were damped a lot, almost to the point where they couldn't be heard.

Checkpoint 4: Show your laboratory assistant your four plots from Step 4 and explain.

Laboratory report:

Submit a listing of your function file and the live script. Include the answers to all questions in the laboratory procedure, all requested plots and all requested data recording.

STEP 1:

Function:

```
function dial_sig = my_dtmf(tone_time, quiet_time, fs, dial_vals)
% dial_sig = my_dtmf(tone_time, quiet_time, fs, dial_vals)
%
% INPUTS:
%   tone_time  - Duration of each DTMF tone in seconds
%   quiet_time - Duration of silence between tones in seconds
%   fs         - Sampling frequency in Hz
%   dial_vals  - Vector of integers (1 to 12) representing keypad
buttons
%                Button 10 = '*', 11 = '0', 12 = '#'
%
% OUTPUT:
%   dial_sig   - Normalized signal vector of DTMF tones for dial_vals
```

```
%                Output is scaled so max(abs(dial_sig)) = 1
   % DTMF frequencies (Hz)
   row_freqs = [697, 770, 852, 941];     % Rows: 1-4
   col_freqs = [1209, 1336, 1477];       % Columns: 1-3
   % Button-to-row/column mapping (row, column)
   % Index represents button value (1 to 12)
   dtmf_map = [1 1; 1 2; 1 3;    % 1, 2, 3
               2 1; 2 2; 2 3;    % 4, 5, 6
               3 1; 3 2; 3 3;    % 7, 8, 9
               4 1; 4 2; 4 3];   % *, 0, #
   tone_len = round(fs * tone_time);        % Samples per tone
   quiet_len = round(fs * quiet_time);      % Samples per quiet segment
   t = (0:tone_len-1) / fs;                 % Time vector for one tone
   quiet_sig = zeros(1, quiet_len);         % Silence vector
   dial_sig = [];                           % Initialize output
   for k = 1:length(dial_vals)
       button = dial_vals(k);

       % Check if button is valid
       if button < 1 || button > 12
           warning('Skipping invalid button value: %d', button);
           continue;
       end

       % Get corresponding row and column indices
       row = dtmf_map(button, 1);
       col = dtmf_map(button, 2);

       % Frequencies for this button
       f1 = row_freqs(row);
       f2 = col_freqs(col);

       % Generate tone as sum of two cosines
       tone = cos(2*pi*f1*t) + cos(2*pi*f2*t);

       % Append tone and quiet to output signal
       dial_sig = [dial_sig tone quiet_sig];
   end
   % Normalize output signal
   if max(abs(dial_sig)) > 0
       dial_sig = dial_sig / max(abs(dial_sig));
   end
end
```

Function Call:

```
x = my_dtmf(.2, .1, 8000, [3, 6, 11]);

plot(x)

sound(x)
```

STEP 2 & 3:

```
tone_len = .25;

quiet_len = .05;

Fs = 8000;

n = 4;

dT = 1/Fs;

x = my_dtmf(tone_len, quiet_len, Fs, [3, 5, 7, 11]);

time = n * (tone_len + quiet_len);

tvec = (0: (time * Fs -1) ) * dT;

plot(tvec, x)

sound(x, Fs)
```

STEP 4:
Moving average filter of length 6:

```
a = 1;

b = [1/6, 1/6, 1/6, 1/6, 1/6, 1/6];

[H,fv] = freqz(b,a,'whole',2048, fs);

plot( fv, abs(H));
```

```
q = filter(b, a, x);

plot(tvec, q);

sound(q, fs)
```

Moving average filter of length 12:

```
a = 1;

b = [1/6, 1/6, 1/6, 1/6, 1/6, 1/6, 1/6, 1/6, 1/6, 1/6, 1/6, 1/6];

[H,fv] = freqz(b,a,'whole',2048, fs);

plot( fv, abs(H));
```

```
q = filter(b, a, x);

plot(tvec, q);

sound(q, fs)
```